

Pipeline e vetorização

Implemente três laços em C para investigar os efeitos do paralelismo ao nível de instrução (ILP):

- 1) Inicialize um vetor com um cálculo simples
- 2) Some seus elementos de forma acumulativa, criando dependência entre as iterações
- 3) Quebre essa dependência utilizando múltiplas variáveis.

Compare o tempo de execução das versões compiladas com diferentes níveis de otimização (O0, O2, O3) e analise como o estilo do código e as dependências influenciam o desempenho.

Código:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define TAMANHO 100000000

double calcular_tempo(struct timespec start, struct timespec end) {
    return (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec)
    / 1e9;
}

int main() {
    double *vetor = (double *)malloc(TAMANHO * sizeof(double));
    if (vetor == NULL) {
        return 1;
    }

    struct timespec start, end;
    double tempo;

    clock_gettime(CLOCK_MONOTONIC, &start);
    for (int i = 0; i < TAMANHO; i++) {
        vetor[i] = i * 2.5;
    }
    clock_gettime(CLOCK_MONOTONIC, &end);
    tempo = calcular_tempo(start, end);
    printf("Tempo do Laco 1 (Inicializacao): %f segundos\n", tempo);
}
```

```

double soma_dependente = 0.0;
clock_gettime(CLOCK_MONOTONIC, &start);
for (int i = 0; i < TAMANHO; i++) {
    soma_dependente += vetor[i];
}
clock_gettime(CLOCK_MONOTONIC, &end);
tempo = calcular_tempo(start, end);
printf("Tempo do Laco 2 (Com dependencia): %f segundos (Soma:
%.2f)\n", tempo, soma_dependente);

double s1 = 0.0, s2 = 0.0, s3 = 0.0, s4 = 0.0;
clock_gettime(CLOCK_MONOTONIC, &start);
for (int i = 0; i < TAMANHO; i += 4) {
    s1 += vetor[i];
    s2 += vetor[i + 1];
    s3 += vetor[i + 2];
    s4 += vetor[i + 3];
}
double soma_independente = s1 + s2 + s3 + s4;
clock_gettime(CLOCK_MONOTONIC, &end);
tempo = calcular_tempo(start, end);
printf("Tempo do Laco 3 (Sem dependencia): %f segundos (Soma:
%.2f)\n", tempo, soma_independente);

free(vetor);
return 0;
}

```

Saída O0:

```

./programa_lento_O0.exe
Tempo do Laco 1 (Inicializacao): 0.205298 segundos
Tempo do Laco 2 (Com dependencia): 0.287684 segundos (Soma:
12499999865005998.00)
Tempo do Laco 3 (Sem dependencia): 0.079350 segundos (Soma:
12499999875000000.00)

```

Saída O2:

```

./programa_medio_O2.exe
Tempo do Laco 1 (Inicializacao): 0.134906 segundos
Tempo do Laco 2 (Com dependencia): 0.075789 segundos (Soma:

```

```
12499999865005998.00)
Tempo do Laco 3 (Sem dependencia): 0.037434 segundos (Soma:
12499999875000000.00)
```

Saída O3:

```
./programa_rapido_O3.exe
Tempo do Laco 1 (Inicializacao): 0.133690 segundos
Tempo do Laco 2 (Com dependencia): 0.075755 segundos (Soma:
12499999865005998.00)
Tempo do Laco 3 (Sem dependencia): 0.034698 segundos (Soma:
12499999875000000.00)
```

Compilação -O0:

Laço 2 (Com dependência - 0.287s): Apresentou o pior tempo de execução. O processador foi forçado a lidar com o risco de dados, gerando bolhas no pipeline. A CPU gastou a maior parte do tempo ociosa, esperando a gravação na memória da variável soma_dependente para poder iniciar a próxima iteração.

Laço 3 (Sem dependência - 0.079s): Mostrou-se aproximadamente 3,6 vezes mais rápido que o Laço 2. O desenrolamento do laço e o uso de acumuladores independentes permitiram que a arquitetura superescalar da CPU entrasse em ação, processando múltiplas adições simultaneamente por ciclo de clock.

O compilador atua apenas como um tradutor ingênuo, convertendo o código C em linguagem de máquina exatamente como foi escrito, sem tentar melhorar a lógica. Todas as variáveis são lidas e gravadas repetidamente na memória principal (RAM). Por conta disso, o tempo de acesso à memória dita o ritmo, tornando todos os laços lentos. O Laço 3 vence aqui apenas porque o seu loop unrolling faz com que a verificação de condição do laço ($i < \text{TAMANHO}$) seja executada menos vezes, economizando ciclos da CPU.

Compilação -O2:

Todos os tempos caíram drasticamente. O Laço 2 passou de 0.287s para 0.075s. Isso ocorreu porque o compilador parou de acessar a memória RAM (ou caches mais lentos) a cada iteração e passou a manter a variável acumuladora diretamente nos registradores internos do processador.

Mesmo com a otimização de registradores para todos, o Laço 3 (0.037s) continuou sendo o dobro mais rápido que o Laço 2. O compilador alocou os quatro acumuladores em quatro registradores distintos, permitindo que a CPU operasse em capacidade máxima paralela.

No nível moderado, o compilador para de acessar a RAM a todo momento e passa a utilizar os registradores para armazenar os acumuladores. O Laço 2 fica mais rápido, mas continua esbarrando na dependência matemática (uma soma deve esperar a outra). Já o Laço 3 atinge um desempenho excelente, pois o compilador aloca os quatro acumuladores (s1, s2, s3, s4) em quatro registradores diferentes, permitindo que a CPU finalmente execute o ILP máximo sem gargalos de memória.

Compilação -O3:

A teoria sugere que o nível -O3 aplicaria a vetorização no Laço 2, transformando-o automaticamente no Laço 3 e igualando seus tempos. Porém, nossos dados mostraram que o Laço 2 continuou lento. Isso ocorre porque o compilador obedece rigorosamente ao padrão IEEE 754 para números de ponto flutuante (double). Como alterar a ordem de uma soma gigante altera os resultados do arredondamento (demonstrado na diferença das somas finais), o compilador se recusa a vetorizar o Laço 2 por segurança, priorizando a precisão matemática exigida pelo programador em detrimento da velocidade.

Laços:

Laço 1: Apenas percorre o vetor e coloca um valor ($i * 2.5$) dentro de cada posição. Preenche o vetor para executar a soma e serve como uma linha de base de tempo de execução para uma operação simples de escrita na memória.

Laço 2: Soma todos os elementos do vetor usando uma única variável "soma_dependente". Força o processador a tropeçar, esse laço foi desenhado propositalmente com um defeito estrutural chamado Dependência de Dados. A iteração 2 não pode começar a somar até que a iteração 1 termine de guardar o resultado. Isso cria as famosas "bolhas" no *pipeline* e mostra como um código mal planejado amarra as mãos de um processador potente.

Laço 3: Soma os elementos avançando de 4 em 4 no vetor, usando quatro variáveis separadas (s1, s2, s3, s4), que só são juntadas no final. Ao usar quatro variáveis independentes, não há mais fila de espera.

Rodada 1 ($i = 0$)

- s1 soma o índice 0 (0)
- s2 soma o índice 1 ($0 + 1$)
- s3 soma o índice 2 ($0 + 2$)
- s4 soma o índice 3 ($0 + 3$)

Rodada 2 (O i pula para 4)

- s1 soma o índice 4 (4)
- s2 soma o índice 5 (4 + 1)
- s3 soma o índice 6 (4 + 2)
- s4 soma o índice 7 (4 + 3)

Rodada 3 (O i pula para 8)

- s1 soma o índice 8 (8)
- s2 soma o índice 9 (8 + 1)
- s3 soma o índice 10 (8 + 2)
- s4 soma o índice 11 (8 + 3)

Em suma, a análise revela que embora compiladores modernos possuam ferramentas poderosas de otimização, eles não conseguem contornar barreiras estruturais ou limitações matemáticas impostas pela linguagem (como a precisão do ponto flutuante). O desempenho de um programa está intrinsecamente ligado à forma como ele é escrito. O Laço 3 prova de forma incontestável que o conhecimento da arquitetura física (como o Paralelismo em Nível de Instrução e o funcionamento dos *pipelines*) é indispensável. Um código arquitetado para evitar dependências de dados sempre superará um código sequencial ingênuo, independentemente do nível de otimização que o compilador tente aplicar.